

A group of people are working in a modern office environment. In the foreground, a man with a beard and glasses is wearing large headphones and looking at a laptop. To his right, another man is also working on a laptop. In the background, a woman is visible, and another man is on the far right. The office has large windows and a modern chandelier with multiple glass shades hanging from the ceiling.

HR AI Advisor

Automated CV screening that **explains itself** — to both sides

Gleb Tretiakov · Vitalii Virronen · Ilya Zalesskii
AaltoAI Hackathon 2026 — AI sovereignty, security & EU data law

recourse.ilia.fi · open source

250 applications. One afternoon.

Generative AI made applying free.

It did not make **reading** free.

Two people are stuck

The HR manager

- Hundreds of CVs per opening
- No time to read them all properly
- Buys a black-box filter → now **legally exposed**
- Cannot answer "why was I rejected?"

The applicant

- Sends 100 applications
- Gets silence, or "we moved forward with other candidates"
- Learns **nothing** — cannot improve, cannot contest
- Reapplies next year with the same CV

Every screening product on the market solves the first problem
and treats the second as someone else's.

Our bet

Tell the rejected candidate
exactly what would have changed the answer.

"One more shipped microservice project would have been enough."

The technique is called **algorithmic recourse** — well studied in papers,
almost absent from shipped hiring products.

What we built

For HR — a screening console

- Upload CVs, get a ranked shortlist
- Author the job as a **100-point budget**
- See which parts of the CV drove the score
- Approve or override every decision

For the candidate — an answer

- Up to **3 independent routes** back to a pass
- Each with a cost and a realistic timeline
- Drafted as an email the manager sends
- Never sent automatically

Open source · self-hostable · runs the whole demo with **no LLM key** if you want it to

The pipeline

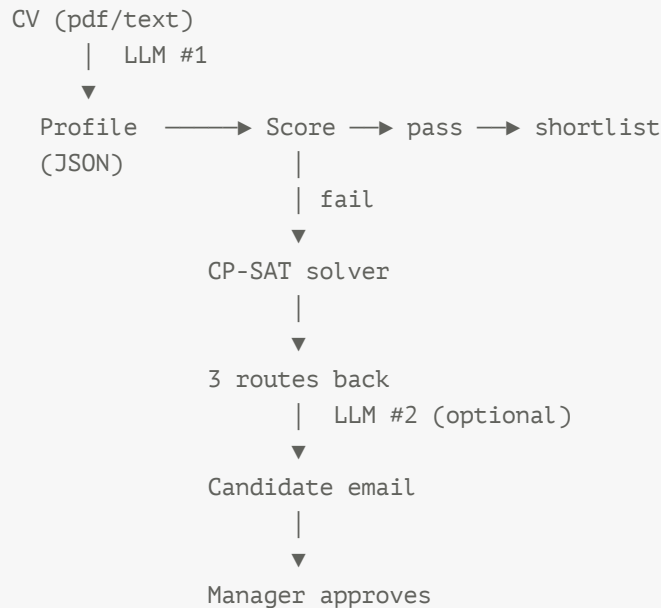
1 • Parse — LLM reads the CV, emits a structured profile. Cached by content hash.

2 • Score — deterministic. Knockouts, then a 100-point budget.

3 • Recourse — CP-SAT solver finds the cheapest changes that flip the outcome.

4 • Explain — code-owned templates; LLM may rephrase, never invent.

5 • Log — hash-chained, append-only audit record per decision.



The LLM appears **twice**. Everything between is deterministic and replayable.

The job is a point budget

HR authors it. The system enforces it. The column sums to **100**, so a score reads as a percentage and the pass mark is a share of the job — not a magic number.

Hard requirements — knockouts

Fail one → rejected. No score traded off against it.

```
skills.python.held == true
experience.software_months >= 12
education.highest_level >= bsc
```

Soft requirements — 100 points

Criterion	Pts
Backend experience	32
Python	16
Kubernetes	11
Microservices projects	10
Cloud platform	9
Education level	6
CI/CD · Docker · IaC · SQL	5+4+4+3

4.1 · Score math

Each criterion has a **cap** — how many solver steps count as full marks — so one step is worth `points // cap`, always an integer.

```
score =  $\sum \min(\text{steps}(\text{feature}), \text{cap}) \times (\text{points} // \text{cap})$ 
```

Mode A — fixed threshold Pass at $\geq 80 / 100$.
Absolute, stable, explainable.

Mode B — top N of pool Pass if you are in the **top 3**. The bar is the 3rd-best score — it *moves*.

Both modes ship. We show both on purpose — see slide "the uncomfortable finding".

Recourse is an optimisation problem

Not a prompt.

Given the candidate's profile, find the **cheapest set of changes** that crosses the bar:

Actionability Every feature is tagged

`actionable` , `conditionally actionable` or `immutable` .

Immutable features are never in a route.

Cost & time Each step has a cost and a typical duration, from the job-family manifest — not invented per candidate.

Causal constraints

$k8s \leq \text{docker}$

$\Delta_{\text{micro}} \leq 2 + \Delta_{\text{backend_mo}}$

You cannot be advised to fake a dependency.

CP-SAT returns up to **3 diverse routes**, each sufficient on its own, sparsity-penalised so short routes win.

Honesty rules, in code

The system must never claim the candidate lacks something it merely **didn't see**.

stated / computed

→ **instruction**

"Add about 6 more months of professional software engineering experience."

denied

→ **instruction that concedes**

"Your CV said you do not have hands-on Kubernetes experience, so gaining it is one way to close this gap."

absent

→ **question**

"Your CV did not mention infrastructure-as-code — if you have it, add it and ask us to re-run the screen."

Verbatim output from the running system.

Keeping the model on a leash

What the LLM writes

- One sentence per change
- That is all

What code writes

- The framing and intro
- Ordering of routes
- Immutable-blocker disclosures
- The closing text

What checks it

A **code-only checker** runs on every generated sentence:

- whitelist of facts it may mention
- no new numbers, no new features
- fails → fall back to the template

No LLM judges another LLM. No "as an AI language model" apology path.

Every screening in this demo ran with `explain=false` — the deterministic path.

The uncomfortable finding

Same candidate. Same CV. Same job.

Threshold mode — bar 80, scored 75

One step:

- *one more microservices project*

cost 6

Ranking mode — bar 91, scored 75,
ranked 5/10

Two steps, and the bar moves if others
improve:

- *one more microservices project*
- *plus hands-on Kubernetes — ~4 months*

cost 15

Recourse is honest under a threshold. Under ranking it is a **moving target** — and we show that rather than hide it.

3 · Law compliance

Hiring AI is **high-risk under Annex III** of the EU AI Act. We designed to the strict reading.

EU AI Act (2024/1689) Art. 86 — right to an explanation of an individual decision. → every decision carries its contributions, its routes and its config version. High-risk duties deferred to 2 Dec 2027 (Digital Omnibus 2026/1744).

GDPR Art. 22 No solely-automated decision with significant effect.
→ **the manager decides**. The system shortlists and drafts; a human sends.

GDPR Art. 13–15 — "meaningful information about the logic involved." → the logic *is* the published point budget.

Data minimisation → protected attributes are never extracted into the profile; the model scores a feature vector, not a person.

NIS2 / sovereignty → self-hostable, two containers, no third-party ATS.

TODO (team): decide how hard we claim compliance vs "designed toward". Framing it as **developmental feedback**, not "the legal reason for rejection", is the safer posture.

Every decision is replayable

An append-only, **hash-chained** log. Each record links to the previous one:

```
record_n.hash = sha256( record_{n-1}.hash + decision_id + timestamp + payload )
```

What's stored profile hash, job version, scorer version, solver version, model id, the routes shown

What it proves edit or delete one line and every hash after it breaks — `make audit` finds the first bad index

Why it matters "show us how this candidate was screened, 14 months ago" has an answer

5 · The console

Three panels, left to right — the same order as the decision.

Uploaded CVs Ranked, best first. Score chip per candidate. Click to open the original PDF or the parsed profile.

Why this match? Hard requirements ticked off, matched requirements, gaps identified with point values. Switch to candidate view.

Candidate email Pre-drafted from the routes. Edit, *Polish with Claude*, then **Send** or **Keep further** — the manager's call.

TODO: drop a fresh screenshot in `deck/public/console.png` and uncomment the image below.

The screenshot in `tmp.pdf` is stale (old 103-point scale) and shows a real CV — don't ship it.

Demo

1. Open the pool — 10 candidates, ranked
2. Open **Aisha** — 75/100, rejected
3. Read the three routes
4. Flip **A** → **B** — watch the advice get more expensive
5. Open the email draft, polish it, *don't* send
6. Open the job editor — move Kubernetes from 11 pts to 0, re-screen

What's real, what's demo

Real

- LLM CV parsing with a content-hash cache
- Deterministic scorer, 100-point budget
- CP-SAT recourse with causal constraints
- Template + checker explanation layer
- Hash-chained audit log
- HR job editor with server-side budget snapping
- 17 test modules, synthetic eval harness

Demo-scale, and we'll say so

- 10 candidates in the pool, one job family
- Ranking mode is deliberately simplified — the bar is just the N-th best score
- Costs and durations are hand-authored in the manifest, not learned
- No bias audit across groups yet
- No ATS integration

Next

Near term

- Recourse-cost fairness audit across groups
- More job families than backend engineering
- Candidate-side re-screen loop ("I added this — recheck me")

The open question

- Does telling people how to pass teach them to **improve** — or to **game**?
- Causal features and dependency constraints are our first answer.
- It needs real data to settle.

HR AI Advisor

Screening that can explain itself — to the manager **and** to the candidate

Gleb Tretiakov · Vitalii Virronen · Ilya Zaleskii

recourse.ilia.fi · open source · AaltoAI Hackathon 2026